



Processo Seletivo INJunior

Desafio Técnico

Exercício 1

Elabore um algoritmo para ler um valor monetário em reais (R\$) e convertê-lo para as moedas euro (€) e dólar americano (US\$), considerando as seguintes taxas de câmbio:

- Taxa de câmbio para euro: €1 = R\$6,10
- Taxa de câmbio para dólar: \$1 = R\$5,70

Imprima na tela o valor em reais inserido pelo usuário e suas conversões para dólar americano e euro.

Exercício 2

Escreva um programa em que o usuário insira os coeficientes a , b e c de uma equação quadrática da forma:

$$ax^2 + bx + c = 0$$

O programa deve:

1. Calcular o discriminante (Δ) pela fórmula: $\Delta = b^2 - 4ac$
2. Avaliar o valor de Δ para determinar o número de raízes reais:
 - Se $\Delta > 0$, exibir as duas raízes reais e distintas.
 - Se $\Delta = 0$, exibir uma raiz real (raiz dupla).
 - Se $\Delta < 0$, informar que a equação não possui raízes reais.
3. Usar a fórmula de Bhaskara para calcular as raízes (caso existam) e imprimir na tela tais raízes:

$$x1 = \frac{-b + \sqrt{\Delta}}{2a} \quad x2 = \frac{-b - \sqrt{\Delta}}{2a}$$



Exercício 3

Escreva um programa que:

1. Solicite ao usuário um número inteiro positivo e calcule o seu fatorial.
2. Pergunte ao usuário se ele quer calcular o fatorial de outro número. Se o usuário digitar "S" ou "s", o programa deve reiniciar e permitir outro cálculo; se digitar "N" ou "n", o programa deve encerrar.
3. Valide a entrada do usuário para garantir que apenas números inteiros positivos são aceitos. Se o usuário digitar um número inválido, exiba uma mensagem de erro e peça a entrada novamente

Exercício 4

Escreva um programa que verifique se uma frase é um palíndromo. Um palíndromo é uma sequência que pode ser lida da mesma maneira da esquerda para a direita e da direita para a esquerda.

Requisitos:

1. Solicite ao usuário uma frase.
2. Remova todos os espaços e ignore a diferença entre letras maiúsculas e minúsculas.
3. Verifique se a frase é igual quando lida ao contrário e informe se ela é um palíndromo ou não.

Dica: Use `toLowerCase()` e `split()`.

Considere o seguinte array de objetos para os exercícios 5 e 6:

```
const booksByCategory = [  
  {  
    category: "Romance",  
    books: [  
      {  
        title: "O Sol Também é Uma Estrela",  
        author: "Nicola Yoon",  
      }  
    ]  
  }  
]
```



```
{
  title: "O Noivo da Minha Melhor Amiga",
  author: "Emily Gin",
},
{
  title: "Orgulho e Preconceito", author:
  "Jane Austen",
},
],
}, {
  category: "Fantasia",
  books: [
    {
      title: "Harry Potter e a Pedra Filosofal",
      author: "J.K. Rowling",
    },
    {
      title: " O Hobbit ",
      author: "J.R.R. Tolkien ",
    },
    {
      title: "Eragon",
      author: "Christopher Paolini",
    },
    {
      title: "O Nome do Vento",
      author: "Patrick Rothfuss",
    },
  ],
}, {
  category: "Suspense",
  books: [
    {
      title: "A Garota no Trem",
      author: "Paula Hawkins",
    },
    {
      title: "O Silêncio dos Inocente s ",
      author: "Thomas Harris",
    },
  ],
}
```



```
    ],  
  }, {  
    category: "Ficção Científica",  
    books: [  
      {  
        title: "Duna",  
        author: "Frank Herbert",  
      },  
      {  
        title: "Neuromancer",  
        author: "William Gibson",  
      },  
      {  
        title: "Fundação",  
        author: "Isaac Asimov",  
      },  
      {  
        title: "1984",  
        author: "George Orwell",  
      },  
      {  
        title: "O Fim da Infância",  
        author: "Arthur C. Clarke",  
      },  
    ],  
  },  
]  
]
```

Exercício 5

Implemente uma função chamada **countBooksInCategory** que conte o número de livros em cada categoria e retorne um objeto com o nome da categoria como chave e a contagem de livros como valor.

Exercício 6

Escreva uma função chamada **authors** que retorne um array com os nomes de todos os autores presentes no objeto **booksByCategory**, desconsiderando autores repetidos.



Exercício 7

Escreva um programa para simular uma fila de espera em uma central de atendimento ao cliente.

O programa deve iniciar exibindo na tela um menu interativo contendo a lista de todos os clientes esperando atendimento, mostrando a posição ao lado do nome (ex.: 1º João, 2º Maria, etc).

O menu deve permitir escolher entre as seguintes opções:

1 - “Novo Cliente”: Adiciona um novo cliente ao final da fila (solicitando o nome do cliente).

2 - “Atender Cliente”: Retira o primeiro cliente da fila e exibe o nome do cliente que está sendo atendido.

3 - “Sair”: Encerra o programa.

O programa só deve ser encerrado ao escolher a opção 3. Caso contrário, ele deve retornar ao menu principal.

Exercício 8

Crie uma função chamada **analisarConsumo** que receba um array de números representando o consumo em kWh de diferentes aparelhos eletrônicos.

Requisitos:

Entrada do Usuário: A função deve solicitar ao usuário (via prompt) um valor numérico que servirá como o limite máximo de consumo para a categoria "Econômico".

Processamento: Para cada valor de consumo no array, determine a categoria:

"Econômico": Se o consumo for menor ao limite inserido.

"No Limite": Se o consumo for igual ao limite inserido

"Acha que é sócio da Enel": Se o consumo for maior que o limite inserido.



Retorno: A função deve retornar um novo array contendo as strings das classificações ("Econômico", "No Limite" ou "Acha que é sócio da Enel"), mantendo a ordem original dos aparelhos.

Saída no Console: Após o processamento, a função deve imprimir uma mensagem para cada aparelho no seguinte formato:

Aparelho 1: Econômico
Aparelho 2: Alto Consumo
(...)

Exercício 9

Escreva uma função chamada **compactarTexto** que receba uma string e compacte caracteres repetidos em sequência.

Requisitos:

- 1 - A função deve percorrer a string caractere por caractere.
- 2 - Deve contar quantas vezes cada caractere aparece em sequência.
- 3 - O resultado deve conter: o caractere seguido da quantidade de repetições.
- 4 - Considere letras maiúsculas e minúsculas como diferentes.

Entrada: "hhhhkkttt"
Saída: "h4k2t2"

Entrada: "aaabbccccc"
Saída: "a3b2c4"

Exercício 10

Você está desenvolvendo um sistema para gerenciar jogadores de um time de esportes.

Cada jogador deve ter as seguintes propriedades:

- Nome (string)
- Idade (número)
- Posição (string)
- Pontuação (número)



Implemente as seguintes funcionalidades:

1. Crie um array vazio chamado **time** para armazenar os jogadores cadastrados.

2. Escreva uma função chamada **adicionarJogador** que:

- Recebe como parâmetros os dados de um jogador (nome, idade, posição e pontuação). Observação: esses dados devem ser inseridos pelo usuário.
- Cria um objeto representando o jogador e o adiciona ao array **time**.

3. Escreva uma função chamada **buscarPorPosicao** que:

- Recebe como parâmetro uma string representando a posição. Observação: esse dado deve ser inserido pelo usuário.
- Retorna todos os jogadores dessa posição.

Se não houver jogadores, exiba uma mensagem apropriada.

4. Escreva uma função chamada **listarTime** que:

- Imprime todos os jogadores cadastrados no **time** no console, um por linha.

5. Escreva uma função chamada **calcularPontuacaoMedia** que:

- Calcula e retorna a pontuação média de todos os jogadores do **time**.

6. O programa deve iniciar exibindo o seguinte menu interativo, que deve permitir ao usuário escolher uma das opções:

- 1 - **Adicionar jogador**
- 2 - **Buscar por posição**
- 3 - **Listar time**
- 4 - **Calcular pontuação média**
- 5 - **Sair**

Boa sorte!

